

Les exceptions en Java

Fiche de révision — hiérarchie, liste et cas d'usage

1. La hiérarchie des exceptions

Tout est basé sur **Throwable**, qui se divise en deux branches : **Error** et **Exception**.

```
Throwable
|-- Error (problemes graves, JVM/systeme, on ne les attrape pas)
|   |-- OutOfMemoryError
|   |-- StackOverflowError
|
|-- Exception
    |-- Exceptions verifiees (checked) -> heritent directement d'Exception
    |-- RuntimeException (non verifiees / unchecked)
```

2. Exceptions vérifiées (checked)

Le compilateur oblige à les gérer (*try/catch* ou *throws*). Elles représentent des situations prévisibles et récupérables, souvent liées à des ressources externes.

Exception	Contexte d'utilisation
IOException	Erreur d'entrée/sortie (fichier introuvable, flux fermé)
FileNotFoundException	Sous-classe d'IOException : fichier absent
SQLException	Erreur lors d'une requête base de données (JDBC)
ClassNotFoundException	Classe introuvable au chargement dynamique (Class.forName)
InterruptedException	Thread interrompu pendant une attente (sleep, wait, join)
ParseException	Erreur de parsing (dates, formats textuels)

Quand les utiliser : quand l'appelant peut et doit réagir (réessayer, afficher un message, changer de fichier). C'est une erreur qu'on peut anticiper.

3. Exceptions non vérifiées (RuntimeException)

Pas obligatoires à gérer : elles représentent souvent des **bugs de programmation**.

Exception	Contexte d'utilisation
NullPointerException	Appel de méthode / accès sur une référence null
ArrayIndexOutOfBoundsException	Index de tableau invalide
IndexOutOfBoundsException	Idem pour les listes
ArithmeticException	Division par zéro (entiers)
ClassCastException	Cast invalide entre types incompatibles
NumberFormatException	Ex : Integer.parseInt("abc")

Exception	Contexte d'utilisation
IllegalArgumentException	Argument invalide passé à une méthode
IllegalStateException	Méthode appelée à un moment inapproprié
UnsupportedOperationException	Opération non supportée (ex. liste immuable)
ConcurrentModificationException	Modification d'une collection pendant son itération
NoSuchElementException	Élément absent (ex. Iterator.next() sans suivant)

Quand les utiliser : pour signaler une erreur de programmation ou une violation de contrat d'API (mauvais argument, état invalide). On les laisse généralement remonter plutôt que de les « avaler ».

4. Les erreurs (Error)

Non destinées à être attrapées : elles signalent que la JVM elle-même est en difficulté.

Error	Contexte
OutOfMemoryError	Plus assez de mémoire (heap plein)
StackOverflowError	Récursion infinie ou trop profonde
AssertionError	Assertion (assert) échouée

5. Créer ses propres exceptions

Checked : hérite d'Exception

```
public class SoldeInsuffisantException extends Exception {
    public SoldeInsuffisantException(String message) {
        super(message);
    }
}
```

Unchecked : hérite de RuntimeException

```
public class ValidationException extends RuntimeException {
    public ValidationException(String message) {
        super(message);
    }
}
```

Règle de choix : Checked si l'appelant a une chance raisonnable de se rétablir (ex. retry, fichier alternatif). Unchecked si c'est un bug ou une condition qui ne devrait normalement jamais arriver.

6. Syntaxe de gestion

```
try {
    // code risque
} catch (IOException e) {
    // gestion spécifique
} catch (SQLException | ClassNotFoundException e) {
    // multi-catch (Java 7+)
} finally {
    // toujours execute (fermeture de ressources)
}
```

```
// try-with-resources (preferable depuis Java 7, ferme automatiquement)
try (FileReader fr = new FileReader("fichier.txt")) {
    // ...
} catch (IOException e) {
    e.printStackTrace();
}
```

7. Bonnes pratiques à retenir

- Ne jamais attraper Exception ou Throwable de façon générique sans raison précise.
- Toujours logger ou relancer (throw) l'exception, ne pas l'avaler silencieusement.
- Privilégier try-with-resources pour la fermeture automatique des flux.
- Ne pas utiliser les exceptions pour du contrôle de flux normal (coûteux en performance).